CLASSPULSE

A teacher-focused interactive classroom and AI learning platform

Project concept

ClassPulse combines a live classroom interaction product inspired by Wooclap with a separate learning workspace for courses, assignments, assessments, attendance, documents, and AI-supported study.

Application	Primary responsibility	Backend
Live Classroom	Live sessions, QR/session-code joining, interactive questions, polls, word clouds, brainstorming, prioritization, timer, slides, random picker	Spring Boot
Learning Workspace	Course and assignment management, question generation, document viewing, student assessment, attendance, AI Study Assistant with RAG	Node.js + Express (MERN)
Shared website	One user-facing website, consistent navigation and login experience, links between the two products	React frontend(s) or shared React app

Important architecture decision

Keep the two backends independently deployable, but do not make two disconnected products. Use stable user IDs, documented APIs, explicit ownership of each data type, and a consistent authentication strategy so a teacher can move from a course to a live session without duplicate accounts.

Prepared as a feature and technology specification for planning, implementation, and project discussions.

1. Product scope and system architecture

ClassPulse has two functional areas with different workloads. The live classroom backend is optimized for low-latency session state and real-time interaction. The MERN backend is optimized for persistent learning content, workflows, files, assessments, and retrieval-augmented AI.

Area	Spring Boot service	MERN service
Owns	Live session, live activity/question, participant presence in session, live answers, activity state, live results	Courses, enrollments, course resources, assignments, submissions, grades/feedback, attendance records, generated question banks, RAG documents/chunks/chat
Main users	Teacher running a session; students joining and answering	Teachers managing learning materials and assessment; students studying and submitting work
Realtime	WebSocket/STOMP or SSE/WebSocket for answers, timers, and results	Mostly REST; optional WebSocket or notifications for updates
Primary database	PostgreSQL recommended for transactional session data; Redis optional for ephemeral state	MongoDB with Mongoose
File/AI workload	Avoid placing the main document-ingestion and RAG pipeline here	File storage, parsers, embedding calls, vector search, LLM orchestration

Suggested high-level request flow

User action	Flow
Sign in	React website → authentication/session layer → identity token used by both APIs
Open course	React → MERN API → MongoDB; course membership and permissions checked by MERN service
Start live session from a course	MERN course page → request/create a Spring Boot session linked to courseId and teacherId → show QR and short join code
Student joins live activity	Student browser → Spring Boot join endpoint → WebSocket channel for answers/results
Ask AI about course files	React → MERN API → authorize user → retrieve permitted document chunks → LLM response with source references

Boundary rule

Each service is authoritative for its own data. The Spring Boot service should not directly read or write MERN's MongoDB collections. The MERN service should not directly manipulate Spring Boot's database. Connect them through documented REST APIs and, where useful, asynchronous events.

2. Spring Boot backend — Live Classroom

This backend contains the Wooclap-style live interaction feature only. It handles the session lifecycle, participant joining, live activity delivery, answers, real-time results, and classroom controls.

Typical end-to-end flow

- **Teacher creates a session:** choose a title, optionally link a course, and prepare a sequence of activities.
- **Class joins:** the system generates a short join code and QR code that encodes the join URL. Students scan or enter the code.
- **Teacher launches an activity:** the activity becomes active on student devices and on the teacher's presentation screen.
- **Students respond:** answers are accepted, validated, timestamped, and associated with the session participant.
- **Results update live:** charts, word clouds, grouped ideas, or rankings refresh in real time.
- **Teacher controls the room:** open/close responses, move to the next activity, start a timer, reveal results, or end the session.
- **Session summary:** save participation and responses so a teacher can review the session later if this is included in the MVP.

2.1 Session creation, QR code and session code

What it does: a teacher creates a live room. ClassPulse generates a unique, human-readable join code plus a QR code for the room URL. Students can join from a phone, tablet, or laptop without needing to find the session manually.

Recommended rules: codes must be unique while active, expire or become invalid when the session ends, and support teacher controls such as allowing anonymous display names or requiring signed-in students. QR codes should contain a URL and non-sensitive join token, not raw student data.

Technology: Spring Boot REST API for creation/joining, a QR library to generate the image, PostgreSQL for session records, and WebSocket for live presence and activity events.

2.2 Live activity lifecycle

Each activity should have a clear state such as **draft** → **ready** → **active** → **closed**. Only one activity needs to be active at a time for the simplest MVP. The server—not just the browser—must decide whether a response is accepted, whether a timer has expired, and whether results are visible.

2.3 Multiple-choice question / live quiz

What it does: the teacher asks a question with selectable answer options. One or multiple options may be correct depending on configuration. Students answer on their devices; after submission, the teacher can reveal a distribution of responses and, for graded quiz questions, the correct answer and score.

Use cases: check understanding, identify misconceptions, run a quick knowledge check, or hold a scored quiz.

Suggested options: question text, options, one/multiple correct answers, optional explanation, points, response limit, timer, shuffle options, anonymous/identified mode, results visibility.

3. Spring Boot — Interactive activity catalogue

3.1 Poll

Purpose: Collect a quick opinion or preference from the whole class using predefined options. Unlike a graded quiz, a poll is normally about what participants think rather than whether an answer is correct.

Teacher workflow: Create a prompt and options, launch it, watch the vote distribution, then close voting and discuss the result.

Student experience: Tap one option and see a confirmation; depending on teacher settings, students may or may not see the live result before voting closes.

How it should behave: Support single-choice voting first. Prevent duplicate votes per participant unless the teacher explicitly enables vote changes. Show counts and percentages after aggregation.

Suggested implementation: Spring Boot activity type + response endpoint; WebSocket pushes updated aggregate counts. Store individual responses only when needed for reports or accountability.

MVP scope: MVP: single-choice poll, live bar chart, close/reopen voting.

3.2 Word cloud

Purpose: Collect short text responses and display frequently submitted words or phrases in a shared visual. Repeated terms appear more prominently so common ideas are easy to notice.

Teacher workflow: Ask a short prompt such as 'What comes to mind when you hear climate change?' and launch the activity.

Student experience: Enter a word or short phrase; the class sees the collective vocabulary form as responses arrive.

How it should behave: Normalize case and whitespace, filter stop words if configured, optionally merge similar terms, moderate inappropriate submissions, and let the teacher hide or remove entries.

Suggested implementation: Text-response API + WebSocket updates + a frontend word-cloud visualization library. Keep raw responses separate from the aggregated display.

MVP scope: MVP: short text input, live word cloud, teacher moderation.

3.3 Open question

Purpose: Collect free-text answers when students need to explain reasoning, share an idea, or reflect in their own words instead of selecting a predefined option.

Teacher workflow: Write a prompt, optionally set answer length and a timer, launch it, and review answers as a list or grouped display.

Student experience: Type a response and submit it; students can see only their own answer or the shared class answers according to settings.

How it should behave: Allow one or multiple responses based on configuration. Add moderation, character limits, optional likes, and optional answer grading for simple expected-answer activities.

Suggested implementation: Store text responses with participant/session/activity IDs; use WebSocket to stream new answers to the teacher screen. Do not expose private student identity if anonymous mode is enabled.

MVP scope: MVP: one response per participant, live response list, teacher hide/delete.

3.4 Random student picker / spin-the-wheel

Purpose: Select a participant at random to encourage broad participation and avoid calling only on volunteers. A wheel can also randomly select a topic or item.

Teacher workflow: Open the picker, select eligible joined participants, spin, and reveal the chosen participant.

Student experience: Students can see the result on the shared screen; the chosen student can answer orally or in a follow-up activity.

How it should behave: Random selection must happen server-side or be auditable enough to prevent the teacher's browser from quietly biasing results. Exclude already-picked students when requested; support reset and manual exclusions.

Suggested implementation: Spring Boot participant list + secure random selection utility + live event broadcast. Store selection history if teachers need fairness tracking.

MVP scope: MVP: select from currently joined participants, avoid repeats until reset.

3.5 Timer

Purpose: Add a time limit to a question, brainstorm, or classroom exercise to create focus and give the class a shared pace.

Teacher workflow: Choose minutes and seconds, start/pause/reset the countdown, and optionally close responses automatically at zero.

Student experience: See the same countdown on the participant screen or main display.

How it should behave: The server should broadcast a start timestamp and duration rather than sending a decrement every second. Clients calculate the displayed remaining time from that reference, while the server enforces the deadline.

Suggested implementation: Spring Boot timer events + WebSocket broadcast; use server time and activity deadline checks to avoid client-clock inconsistencies.

MVP scope: MVP: start, pause, reset, expiry event, optional auto-close.

3.6 Slides / content slide

Purpose: Show instructions, a quote, links, bullet points, or lesson context between interactive questions. It helps a teacher alternate explanation and participation.

Teacher workflow: Add a slide to the session sequence, enter a title and content, then display it between activities.

Student experience: View the slide at the same time as the class; move forward when the teacher advances.

How it should behave: Keep slide content distinct from interactive questions. Synchronize current slide/activity index for all connected clients. For MVP, use text and images; complex PowerPoint import can be later.

Suggested implementation: Spring Boot session sequence and slide model; React presentation renderer; optional object storage for images.

MVP scope: MVP: title, text, bullets, optional image, teacher-controlled next/previous.

3.7 Brainstorming

Purpose: Collect many ideas from a group and organize them into a shared visual. This is for idea generation and grouping, rather than a single correct answer.

Teacher workflow: Create a prompt and optional categories/columns, launch the activity, then review and organize contributions.

Student experience: Submit ideas; the shared view shows contributions in a list, cards, or categories.

How it should behave: Support category names, moderation, moving ideas between categories, and optional likes/votes if desired. Prevent untrusted content from rendering as executable HTML.

Suggested implementation: Spring Boot for activity/idea records and permissions; WebSocket for new ideas and edits; React for a shared board-like UI.

MVP scope: MVP: prompt, multiple ideas per student, editable categories, teacher moderation.

3.8 Prioritization

Purpose: Make trade-offs visible by asking students to allocate points across several options or rank them. This reveals the group's priorities instead of just collecting a yes/no response.

Teacher workflow: Enter the prompt and items to prioritize, launch the activity, and inspect the aggregate ranking.

Student experience: Allocate a fixed budget of points across options (for example, 100 points total) or rank items, depending on the chosen mode.

How it should behave: Validate that the total points do not exceed the allowed budget, show aggregate totals and ranking, and clearly explain whether higher scores mean higher priority.

Suggested implementation: Spring Boot validation and aggregation; React sliders/point allocation interface; WebSocket updates for results.

MVP scope: MVP: 100 points distributed across 3–8 items; aggregate ranking after submission.

4. Spring Boot — Supporting capabilities

These are not separate question types, but they make the live classroom feature feel complete and reliable.

Capability	What it does	Implementation notes
Teacher live dashboard	Shows joined count, active activity, response count, timer state, and controls.	React view backed by Spring Boot REST + WebSocket events.
Participant list / presence	Tracks who joined and whether they are currently connected.	Use join/heartbeat/disconnect events; do not treat temporary network disconnect as proof the student left permanently.
Results view	Displays poll charts, answer distributions, word clouds, idea boards, and prioritization rankings.	Server computes or validates aggregates; frontend renders visualizations.
Session sequence	Lets teachers prepare multiple activities and move through them in order.	Store ordered activity IDs and a current index; validate all transitions server-side.
Anonymous mode	Lets a teacher show responses without exposing student names to the class.	Keep identity/authorization data private; anonymize only display, not necessarily audit records.
Session report	Allows a teacher to review attendance and activity responses after class.	Save event/response data and define retention/export rules.
Reconnect handling	Allows students to recover after refresh or a weak connection.	Use a short-lived participant token and fetch current session/activity state after reconnect.
Moderation and safety	Lets the teacher hide inappropriate text, close submissions, or remove a participant.	Authorization checks, audit log, input validation, rate limits.

Suggested Spring Boot stack

Layer	Recommended choice	Reason
Language/framework	Java + Spring Boot	Separate, strongly structured service for live session logic.
API	Spring Web / REST	Create sessions, join, manage activities, retrieve reports.
Realtime	Spring WebSocket + STOMP (or WebSocket with a simple protocol)	Push answers, counts, activity changes, and timer events.
Database	PostgreSQL + Spring Data JPA	Transactional session/activity/response records and relational integrity.
Ephemeral state (optional)	Redis	Presence, short-lived room state, rate limiting, or multi-instance pub/sub if needed.
Security	Spring Security + JWT/OIDC validation	Protect teacher controls and validate user identity.
Testing	JUnit 5, Spring Boot Test, Testcontainers	Unit, integration, database, and permission testing.
API docs / deployment	OpenAPI/Swagger, Docker	Document and package the service for consistent deployment.

5. MERN backend — Learning Workspace

The MERN side is the broader teacher-focused learning environment. It stores courses and course materials, helps teachers prepare questions and assignments, tracks student work and attendance, and answers student questions using approved course documents.

5.1 Course and Assignment Management

Purpose: give each teacher a place to organize classes, resources, assignments, due dates, submissions, marks, and feedback.

Teacher features

- Create, edit, archive, and organize courses; set course title, description, subject, level, and term.
- Enroll students through invite code, class join link, or controlled roster import.
- Create course modules/topics and attach PDFs, DOCX files, slides, links, and other approved resources.
- Create assignments with instructions, attachments, due date, total marks, allowed submission types, and late-submission policy.
- View submission status (not submitted, submitted, late, graded), grade work, and provide written feedback or rubric-based marks.
- Notify students about new resources, assignments, approaching deadlines, marks, and teacher feedback.

Student features

- View enrolled courses, modules, resources, assignments, due dates, and submission status.
- Upload or submit work, replace a submission if policy allows, and see a timestamped receipt.
- View grades, rubric feedback, teacher comments, and resubmission requests.
- Receive in-app notifications for deadlines and course updates.

Core data: User, Course, Enrollment, CourseModule, Resource, Assignment, Submission, Grade/Feedback, Notification, AuditEvent.

Recommended implementation: React + Vite, Node.js, Express, MongoDB + Mongoose, JWT/OIDC authentication, Zod/Joi validation, and private object storage for uploaded files.

5.2 Question Generation

Purpose: help teachers turn a topic, learning objective, notes, or uploaded material into a draft question bank, saving preparation time. This is a preparation feature in the MERN app; it is separate from the Spring Boot live question runtime.

Teacher workflow: choose course/topic, provide source text or select an uploaded resource, choose question types, difficulty, number of questions, and learning level, then generate, review, edit, and save the questions.

Question types for the first release: multiple choice with answer key and explanation; true/false; short answer; open-ended discussion prompt. Later: matching, fill-in-the-blank, sorting, and question formats used by live sessions.

Quality and safety requirements: show the source used, mark content as AI-generated draft, require teacher review before publishing, validate the answer key, avoid inventing claims unsupported by source material, and allow edit/regenerate/delete.

Suggested stack: Express API, LLM provider API, optional RAG retrieval from course documents, schema-validated JSON output, MongoDB question bank, and background jobs for large documents.

6. MERN — Document viewer, assessment, and attendance

6.1 Slides, PDF viewer, and DOCX viewer

Purpose: let teachers publish learning materials and let students read them inside ClassPulse without repeatedly downloading files or switching applications.

PDF: embedded viewer with page navigation, zoom, search where supported, download permission, and page-level references for the AI assistant.

DOCX: render the document as readable HTML or convert it to PDF for viewing. Preserve headings, lists, and tables as much as practical; do not assume a browser can natively render a DOCX file.

Slides: support uploaded PPTX/PDF slide decks. For a practical first version, convert to PDF or image pages for consistent viewing. Native slide editing and perfect animations are not MVP requirements.

Teacher controls: upload, rename, reorder, group into modules, replace, archive, and set visibility. **Student controls:** open, search/navigate, zoom, and download only when permitted.

Implementation: store file metadata in MongoDB and file bytes in private object storage (S3-compatible storage or equivalent). Use signed URLs, MIME/size validation, malware scanning where available, and server-side access checks. PDF.js can power PDF viewing; DOCX can be rendered via a conversion or parsing pipeline.

6.2 Student Assessment

Purpose: let teachers evaluate student understanding and progress through quizzes, assignment rubrics, scores, feedback, and course-level summaries.

- Create a quiz or assessment from a saved question bank; set marks, duration, attempts, and availability window.
- Auto-grade objective formats such as multiple choice and true/false; keep teacher grading for essays and subjective answers.
- Show feedback and results according to teacher release settings.
- Display per-student progress and course-level summaries; identify questions many students missed.
- Allow rubric-based marking and comments for assignments.

Important separation: MERN assessment is the persistent assessment and gradebook workflow. Spring Boot is the live classroom response engine. If live quiz responses should count toward a formal grade, define an explicit integration/export process rather than sharing databases.

6.3 Student Attendance

Purpose: maintain a reliable attendance record for courses or individual classes, including teacher review and correction.

Suggested ways to mark attendance: teacher marks the roster manually; students check in with a time-limited class code/QR; or attendance is proposed from a linked Spring Boot live session's participant list and then confirmed by the teacher.

Attendance states: present, absent, late, excused. Store the class date, student, status, check-in time, source, and who edited the record.

Integrity: QR/code check-in alone does not prove physical presence. Use expiry, one check-in per student per class, server timestamps, teacher override, and audit history. Avoid collecting continuous location unless there is a clear need and appropriate consent.

Implementation: MERN AttendanceRecord collection and attendance APIs; optional Spring Boot session integration via API. The MERN service owns the official course attendance register.

7. MERN — AI Study Assistant with RAG

Purpose: students ask questions about their own course materials and receive answers grounded in approved lecture notes, PDFs, DOCX files, and slides, with references to the supporting source.

What RAG means

Retrieval-Augmented Generation (RAG) first retrieves relevant passages from authorized course documents, then gives those passages to a language model to draft a response. The goal is to answer from the class material rather than rely only on the model's general knowledge.

Core user features

- **Ask course questions:** e.g., 'Explain normalization from Week 3 notes.'
- **Source citations:** show the document name and page/section or chunk reference used for each answer.
- **Follow-up chat:** ask clarifying questions while preserving the relevant conversation context.
- **Explain at different levels:** simpler explanation, step-by-step breakdown, example, or quick recap.
- **Study aids:** generate flashcards, summaries, and practice questions from authorized content (phase 2).
- **Honest fallback:** if the retrieved material does not support an answer, say that the course sources do not provide enough information instead of pretending it does.

RAG processing pipeline

Step	What happens
1. Upload	Teacher uploads a file to a course resource; backend checks file type, size, permission, and processing status.
2. Extract	Parse text from PDF/DOCX/PPTX. OCR may be needed for scanned PDFs or image-only pages.
3. Chunk	Split text into manageable passages while preserving metadata such as courseId, resourceId, filename, page, heading, and chunk index.
4. Embed	Convert each chunk into a vector using an embedding model.
5. Index	Store vectors and metadata in MongoDB Atlas Vector Search or a dedicated vector database.
6. Retrieve	For a student question, search only the documents that student is authorized to access; retrieve the best matching passages.
7. Generate	Send the question plus retrieved passages to the LLM with instructions to ground the answer in those passages.
8. Cite and log	Return answer and source references; log document IDs, latency, and errors without storing unnecessary sensitive content.

8. MERN stack and shared technology choices

Layer / feature	Recommended technology	Reason / notes
Frontend	React + Vite; React Router; Tailwind CSS or CSS Modules	Build teacher and student dashboards, course pages, viewers, question editor, and AI chat.
MERN API	Node.js + Express	REST endpoints for courses, assignments, documents, assessment, attendance, question generation, and RAG.
MERN database	MongoDB + Mongoose	Flexible documents and metadata; enforce indexes and validation for critical fields.
Identity	Shared OIDC provider/JWT validation; bcrypt only if managing passwords directly	Prefer one identity provider so the same teacher/student identity can access both services.
File storage	Private S3-compatible object storage; local storage for development only	Keep large file bytes out of MongoDB; use signed URLs and authorization checks.
PDF viewer	PDF.js	Page rendering, zoom, and navigation; citation UI can link to a page.
DOCX / PPTX	Server-side parser/converter or trusted document rendering service	Browsers do not reliably render DOCX/PPTX directly with original formatting.
Question generation / RAG	LLM API + embedding model + MongoDB Atlas Vector Search or vector DB	Keep model provider behind an internal service layer to make it replaceable.
Background processing	Redis + BullMQ (optional)	Process large files, embeddings, OCR, exports, and bulk question generation asynchronously.
Validation / security	Zod/Joi/express-validator, rate limiting, secure headers	Validate inputs and apply role/course-level authorization on every request.
Testing	Vitest/Jest, Supertest, Playwright	Test API rules, upload workflows, permissions, and important end-to-end user journeys.
Operations	Docker, environment variables/secrets manager, structured logs	Reproducible deployment, safe configuration, and troubleshooting.

9. Recommended data ownership

Data	Authoritative owner	Notes
User identity	Shared identity provider / auth layer	Both APIs validate the same stable subject/user ID; do not duplicate user accounts.
Course, enrollment, resources	MERN	MERN owns who is enrolled and which materials are available.
Assignments, submissions, grades	MERN	Official coursework and gradebook.
Attendance register	MERN	Can import/derive suggested attendance from a Spring Boot session, then teacher confirms.
Document chunks and AI chats	MERN	All retrieval must be scoped to authorized course/resource IDs.
Live session, live activity, answers	Spring Boot	Spring Boot owns live-room lifecycle and realtime response state.
Cross-service references	Both store external IDs only	For example, a Spring Boot session may store courseId and teacherId from MERN/identity, without copying the full course record.

10. Security and reliability requirements

- **Authorization on the server:** verify teacher ownership and student enrollment on every course/resource/assignment request; do not trust IDs sent by the browser.

- **RAG access isolation:** filter retrieval by authorized course/resource IDs before sending any text to an LLM. Never retrieve from another class just because a chunk is semantically similar.
- **File security:** restrict file types and sizes, use private storage, verify access before downloads, and avoid executing or rendering uploaded HTML/scripts.
- **AI quality:** treat document text as untrusted data, defend against prompt injection in files, cite sources, and use an explicit not-enough-evidence response.
- **Live session integrity:** server validates activity state, duplicate submissions, timer expiry, join-code validity, and teacher-only controls.
- **Privacy:** collect only required student data, define retention/deletion rules, and limit who can view individual responses, attendance, and grades.
- **Observability:** log errors and request IDs, but avoid logging passwords, access tokens, or unnecessary student document contents.
- **Resilience:** handle reconnects, retries, duplicate background jobs, model/API rate limits, and partial document-processing failures.

11. Build plan — recommended order

Phase	Build	Exit criteria
1. Foundation	Shared auth, React shell, role model, service-to-service API conventions, Docker development setup.	Teacher and student can sign in and reach the correct dashboard; both APIs validate the same identity.
2. Course basics	Courses, enrollment, resources, assignment creation/submission, teacher feedback.	A student can join a course, view a resource, submit an assignment, and see feedback.
3. Live classroom MVP	Spring Boot session creation, QR/code join, multiple-choice, poll, open question, live results, timer.	Teacher can run a full 10–30 student session with reconnect handling and server-side validation.
4. More live activities	Word cloud, random picker, slides, brainstorming, prioritization.	Each activity works on teacher and student screens and supports moderation and reset/close controls.
5. Documents and question generation	PDF/DOCX/PPTX viewing, question generator, review/edit/save question bank.	Teacher can create a question bank from a topic or selected material and review before use.
6. Attendance and assessment	Attendance register, quizzes, gradebook, rubrics, reports.	Attendance edits are auditable; objective questions auto-grade correctly; subjective grading stays teacher-controlled.
7. RAG assistant	Parsing, chunking, embeddings, vector search, source citations, grounded answer policy.	Answers cite the correct course resources, access is isolated by enrollment, and unsupported answers fall back safely.
8. Hardening	Accessibility, load tests, monitoring, backups, data retention, security review.	Key journeys are tested; upload and live-session abuse cases are handled; recovery process is documented.

12. Example API responsibilities

Service	Example endpoint	Purpose
Spring Boot	POST /api/live-sessions	Teacher creates a session; returns session ID, join code, and QR URL.
Spring Boot	POST /api/live-sessions/{code}/join	Student joins a session and receives participant/session state.
Spring Boot	POST /api/live-sessions/{id}/activities	Create a live activity in the session sequence.
Spring Boot	POST /api/activities/{id}/responses	Submit a response; server validates active state, deadline, and participant.
Spring Boot	GET /api/live-sessions/{id}/results	Return permitted aggregate results or teacher report.
MERN	POST /api/courses	Create a course for the authenticated teacher.
MERN	POST /api/courses/{id}/assignments	Create an assignment with due date, marks, and submission rules.
MERN	POST /api/assignments/{id}/submissions	Submit student work and record submission timestamp.
MERN	POST /api/resources/{id}/process	Queue document extraction and indexing for RAG.
MERN	POST /api/ai/study-chat	Ask a course-scoped question and return answer plus source citations.
MERN	GET /api/courses/{id}/attendance	Get the authorized course attendance register.

13. Minimum viable product recommendation

Avoid implementing every feature at once. Start with identity, course enrollment, and a small but complete live session experience. The most demonstrable first release is: teacher creates a course, creates a live session, students join by QR/code, teacher launches a multiple-choice question or poll, results update live, and the session can be reviewed. Then add assignments and document viewing, followed by question generation, attendance, and finally RAG.

Definition of done for each feature

- Teacher and student workflows both exist and are tested.
- Permissions are enforced by the backend, not only by hidden UI controls.
- Loading, empty, error, retry, and reconnect states are handled.
- Important actions have validation and tests.
- Data ownership and API contract are documented.
- A short demo can show the full feature working end-to-end.

14. Source notes and terminology

The Wooclap-inspired feature descriptions in the live classroom section were researched against the official Wooclap features page and official Wooclap Help Center pages. ClassPulse-specific implementation choices, data models, security requirements, and build order are recommendations rather than claims that Wooclap implements them exactly this way.

Official references consulted

1. Wooclap — Features and Integrations: https://www.wooclap.com/en/features/
2. Wooclap Help Center — Quick Questions: https://docs.wooclap.com/en/articles/3548119-how-to-use-quick-questions
3. Wooclap Help Center — Prioritisation: https://docs.wooclap.com/en/articles/8344712-prioritisation
4. Wooclap Help Center — Open question: https://docs.wooclap.com/en/articles/8336932-open-question
5. Wooclap Help Center — Question Builder: https://docs.wooclap.com/en/articles/8774298-question-builder
6. Wooclap Help Center — Event features and settings: https://docs.wooclap.com/en/collections/2357545-event-features-and-settings

Glossary

Term	Meaning in ClassPulse
MERN	MongoDB, Express, React, Node.js.
REST API	HTTP endpoints used by the frontend or another service to request or change data.
WebSocket	A persistent connection used to push live events between server and connected browsers.
RAG	Retrieval-Augmented Generation: retrieve relevant source passages, then generate a grounded response.
Embedding	A numerical representation of text used to find semantically similar passages.
Vector search	Search based on semantic similarity between embeddings rather than exact keyword matches alone.
RBAC / authorization	Rules determining which user may perform an action on a particular resource.
MVP	Minimum Viable Product: the smallest complete release that proves the main user journey.

Final architecture principle

Keep the separation clear: Spring Boot runs the live classroom; MERN runs the learning workspace and AI study tools. Connect them through APIs and shared identity—not shared database access. This keeps each backend easier to build, test, explain in interviews, and extend later.